Project Documentation

Date	29 September 2026
Team ID	SWTID-2026-2630
Project Name	AI Fit Track API
Maximum Marks	20 Marks

1. Team Details

S.No	Name	Role	Email
1	Thamarai Selvan M	Team Lead	thamaraiselvanm728@gmail.com
2	Bhavesh K M	Member	bbravobhavesh@gmail.com
3	Vishwa R	Member	rvishwa409@gmail.com
4	Mariya Akash A	Member	mariyaakash00@gmail.com
5	Deepak Vishal S	Member	deepakvishal136@gmail.com

Team ID	SWTID-2026-2630
Project Title	AI Fit Track – AI-Powered Workout Tracking & Fitness Coaching Backend
GitHub Repository	https://github.com/thamaraiselvan6/fittrack_team_projt
Demo Video	Demo recording (Google Drive)

2. Project Description

AI Fit Track is a secure, RESTful backend service that replaces manual, paper-based workout logging with a centralized, authenticated fitness-tracking API. Users register and log in with JWT-protected accounts, perform full CRUD and search operations on their workout history (name, category, duration, calories burned, date), and receive two forms of Generative-AI assistance powered by Google Gemini: a personalized workout recommendation built from their age, fitness goal and experience level, and a fitness insight summarizing patterns in their own logged workout statistics. The system is built with Node.js, Express.js and MongoDB (via Mongoose), following a modular MVC-inspired structure that separates configuration, routing, controllers, middleware, models and AI services.

3. Scenario-Based Case Study

Persona: Rahul, a fitness enthusiast who trains consistently but has always logged his workouts by hand in notebooks and spreadsheets. His records are scattered, occasionally inconsistent, and give him no way to know if he is actually progressing or what to train next. After adopting AI Fit Track, Rahul registers once, logs every session going forward through a single authenticated API, searches his own history instantly by name or category, and periodically requests an AI-generated workout recommendation (based on his age, goal and experience) and a fitness insight (based on his accumulated workout statistics) – turning what used to be a pile of scattered notes into a structured, queryable, and intelligently-guided training log.

4. Software & Hardware Requirements

Category	Requirement
Operating System	Windows / macOS / Linux (development-machine agnostic)
Runtime	Node.js v16 or later
Database	MongoDB (local instance for this cycle: `mongodb://localhost:27017/aifittrack`)
Framework / Libraries	Express.js, Mongoose, jsonwebtoken, bcryptjs, dotenv, cors, nodemon
AI Service	Google Gemini API (accessed via a dedicated `geminiService.js`)
Development Tools	VS Code, Thunder Client / Postman, Git & GitHub
Minimum Hardware	4 GB RAM, dual-core processor, internet connection (required for Gemini API calls)

EPIC 1 – System Architecture

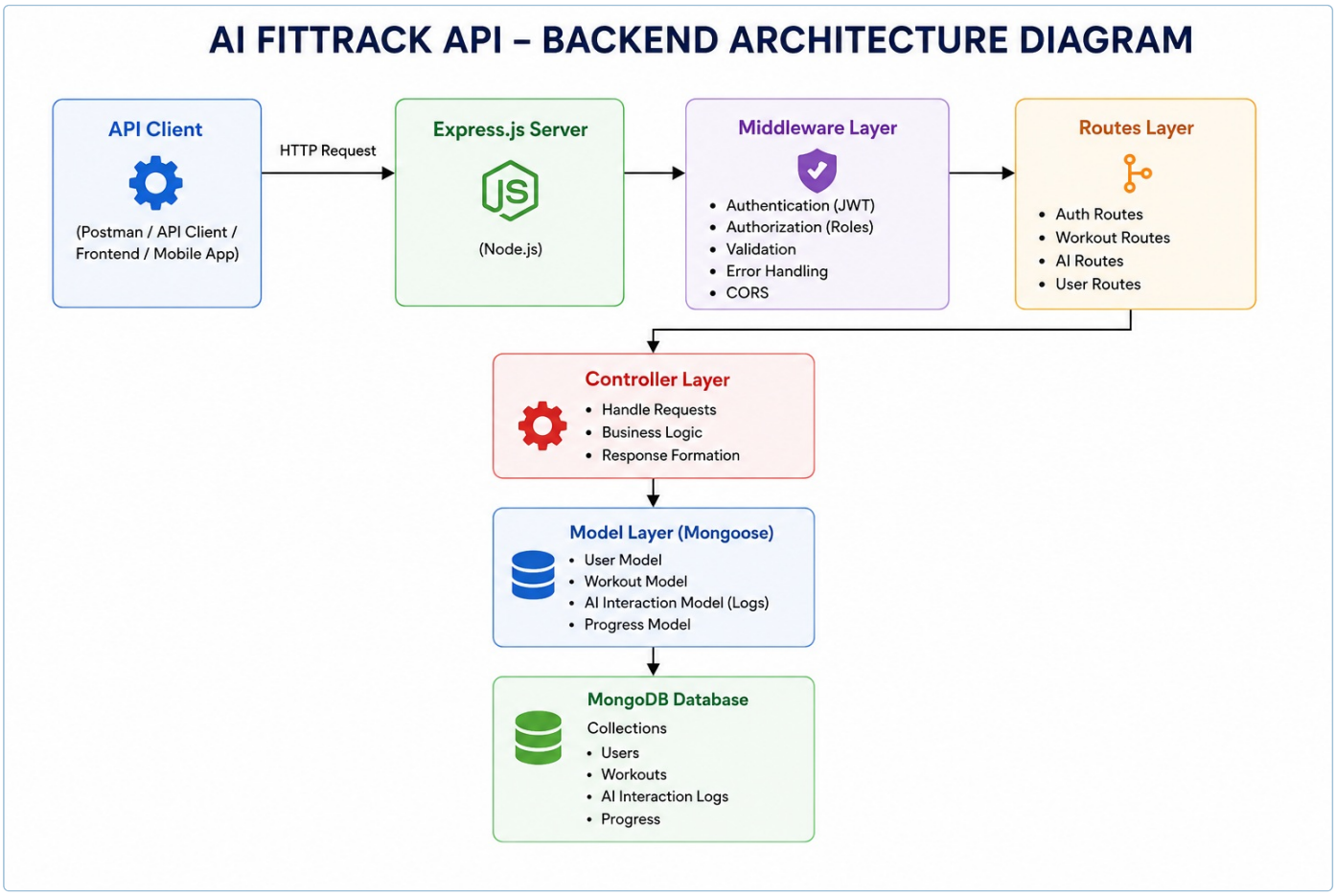


Fig. 1 – Layered architecture: Client → Express Server → Auth Middleware → Routes → Controllers → Service/Model → MongoDB

ER Diagram – Data Model

Currently implemented model: two collections – **User** (_id, name, email [unique], password [bcrypt-hashed], createdAt, updatedAt) and **Workout** (_id, user [reference to User], workoutName, category [enum], duration, caloriesBurned, workoutDate, createdAt, updatedAt) – related one-to-many: one User has many Workout documents, each Workout belonging to exactly one User.

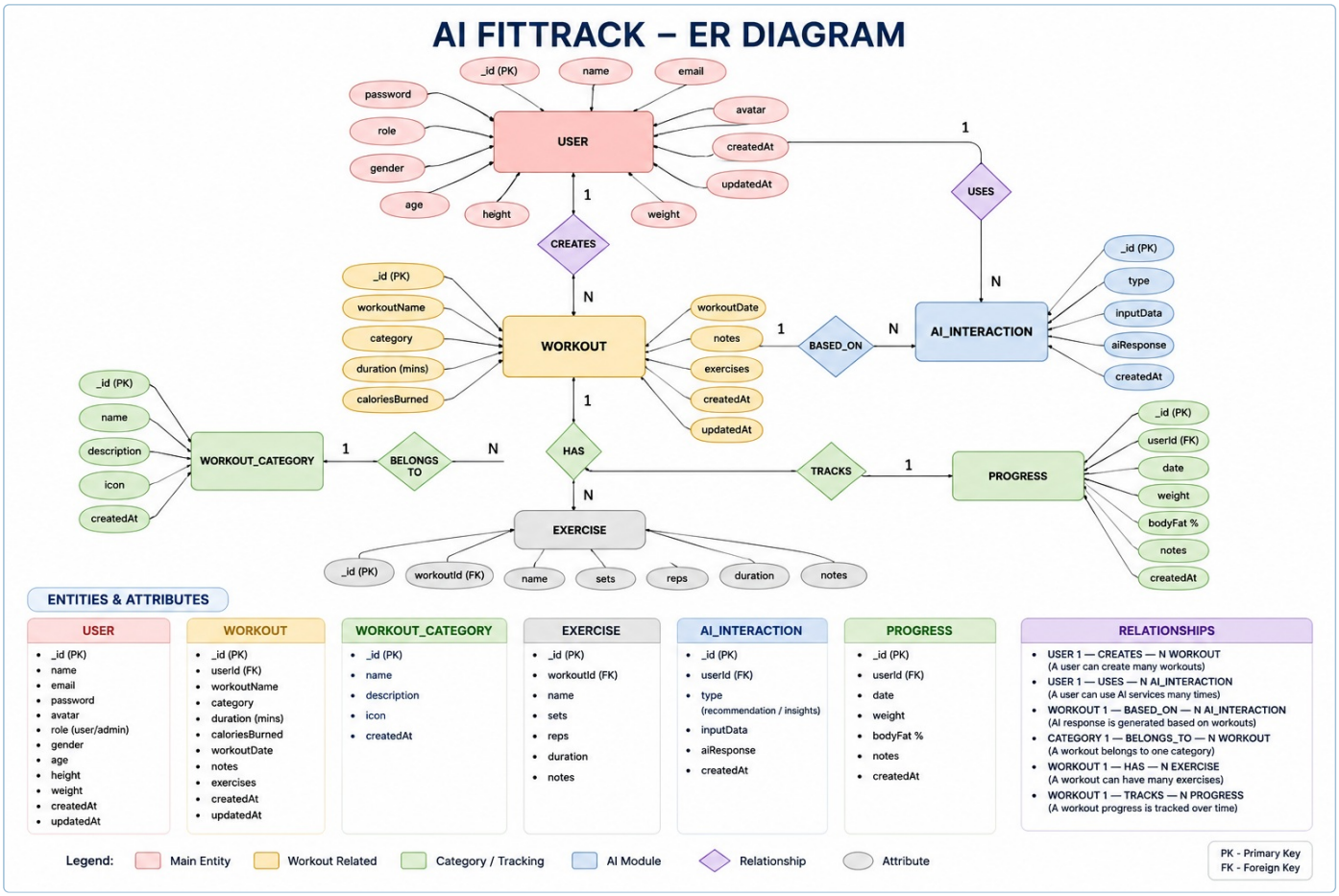


Fig. 2 – Full ER diagram as designed by the team, including the currently-implemented User–Workout model plus additional entities (WORKOUT_CATEGORY, EXERCISE, AI_INTERACTION, PROGRESS) planned for the Scalability & Future Plan roadmap (see Phase 8) rather than built in this cycle.

Key Features

- Secure JWT-based authentication with bcrypt password hashing
- Full CRUD on workout records, scoped to the authenticated user
- Workout search by name/category
- Google Gemini-powered personalized workout recommendation
- Google Gemini-powered fitness insights from logged workout history
- Centralized error handling with structured JSON responses
- Modular, RESTful, maintainable codebase (config/controllers/middleware/models/routes/services)

Roles and Responsibilities

Role	Capabilities
Authenticated User (single role)	Register, Login, View Profile, Add Workout, Get Workouts, Search Workouts, Update Workout, Delete Workout, request AI Recommendation, request Fitness Insights – all permissions apply uniformly, as there is no admin/multi-role separation in this cycle.

User Flow

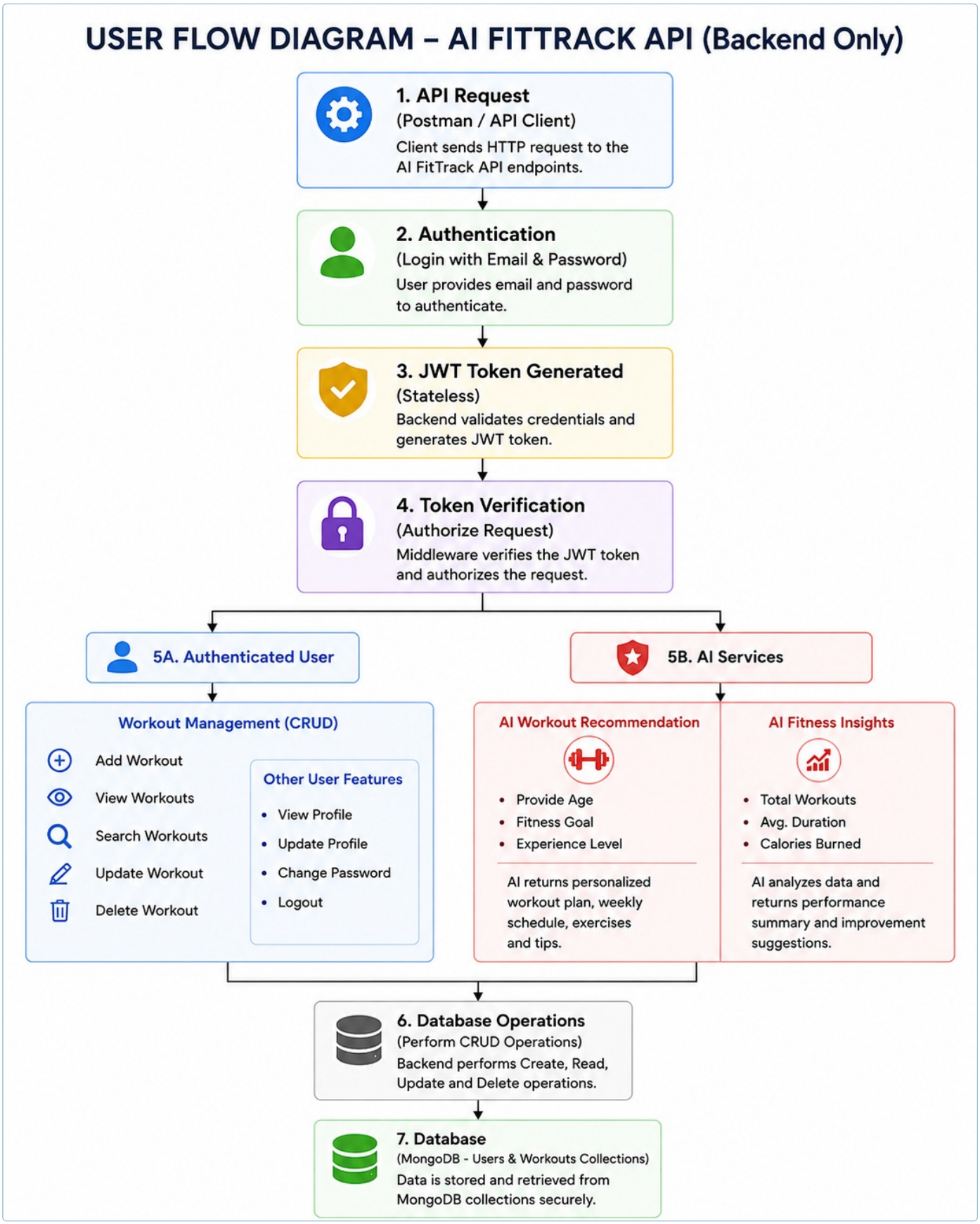


Fig. 3 – End-to-end user flow: Register/Login → JWT issued → Manage Workouts (CRUD/Search) → Request AI Recommendation/Insights → Receive Gemini-generated response

MVC Pattern

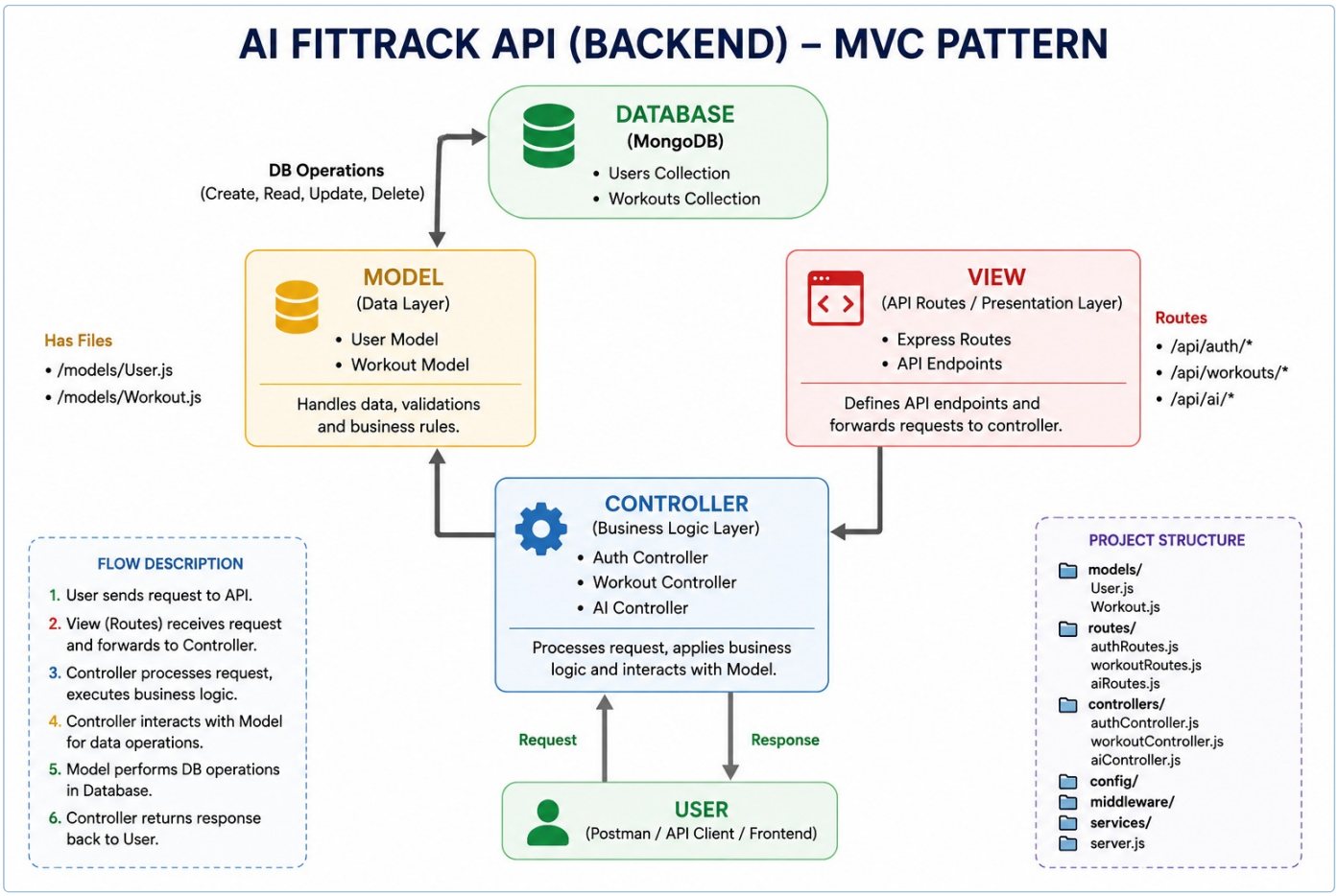


Fig. 4 – Models (User, Workout) hold schema/validation; Controllers hold request logic and orchestrate the Gemini service; the JSON response itself serves as the "View" for this API-only project.

EPIC 2 – Project Setup

Initialized the Node.js project (`npm init`), installed Express, Mongoose, jsonwebtoken, bcryptjs, dotenv, cors and nodemon, and scaffolded the `src/` folder structure (config, controllers, middleware, models, routes, services) plus a placeholder `client/` folder for a future frontend. Configured `.env` for `PORT` , `MONGO_URI` , `JWT_SECRET` and `GEMINI_API_KEY` , excluded via `.gitignore` .

EPIC 3 – Backend Development

Implemented `authController.js` (register/login/profile), `workoutController.js` (full CRUD + search), and `aiController.js` (recommendation + insights), each paired with its own route file and protected, where required, by the shared JWT `auth.js` middleware. Built `geminiService.js` to isolate all Google Gemini prompt construction and response parsing away from the controllers, and a shared `errorHandler.js` middleware for consistent error responses across every route.

EPIC 4 – Database Configuration

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

config/db.js – Mongoose connection with error logging and process exit on failure

src > models > JS User.js > ...

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Please add a name'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Please add an email'],
      unique: true,
      trim: true,
      lowercase: true,
      match: [
        /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
        'Please add a valid email address',
      ],
    },
    password: {
      type: String,
      'Please add a password'],
      minlength: [6, 'Password must be at least 6 characters'],
    },
  },
  {
    timestamps: true,
  }
);

// Pre-save password hashing hook
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) {
    return next();
  }
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Compare password method
userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

0626T085143Z-3-001\FitTrack\src\services

models/User.js – schema with unique email, bcrypt pre-save hook and matchPassword method

src > models > JS Workout.js > workoutSchema > category > enum > values

```
const mongoose = require('mongoose');

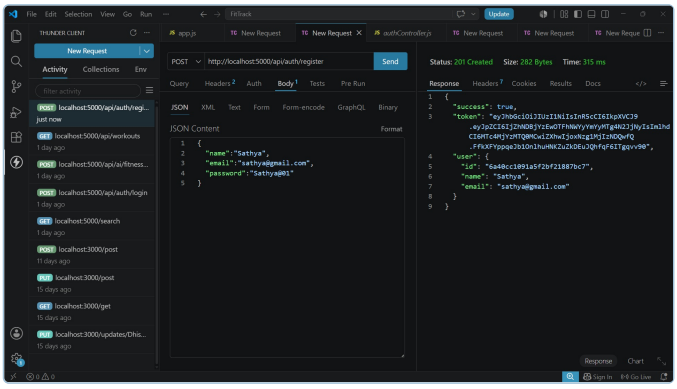
const workoutSchema = new mongoose.Schema(
  {
    workoutName: {
      type: String,
      required: [true, 'Please add a workout name'],
      trim: true,
    },
    category: {
      type: String,
      required: [true, 'Please select a workout category'],
      enum: {
        values: [
          'Cardio',
          'Strength Training',
          'Yoga',
          'Running',
          'Cycling',
          'Walking',
        ],
        message: '{VALUE} is not a supported workout category',
      },
    },
    duration: {
      type: Number,
      required: [true, 'Please add the workout duration in minutes'],
      min: [1, 'Duration must be at least 1 minute'],
    },
    caloriesBurned: {
      type: Number,
      required: [true, 'Please specify calories burned'],
      min: [0, 'Calories burned cannot be negative'],
    },
    workoutDate: {
      type: Date,
      default: Date.now,
    },
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
  },
  {
    timestamps: true,
  }
);

module.exports = mongoose.model('Workout', workoutSchema);
```

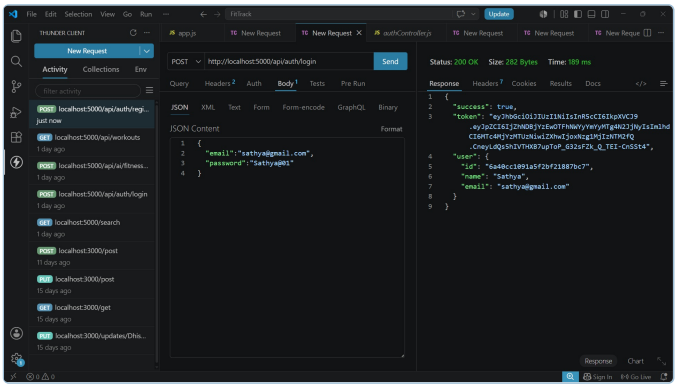
models/Workout.js – schema with a User reference, category enum, duration/caloriesBurned validators and timestamps

The User schema's `pre('save')` hook hashes the password with bcrypt only when it has been modified, and its `matchPassword` instance method compares a plaintext login attempt against the stored hash – passwords are never stored or returned in plaintext at any point.

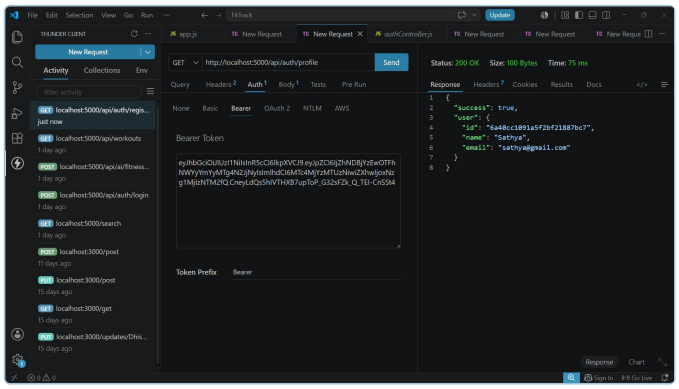
EPIC 5 – Execution & API Testing Evidence



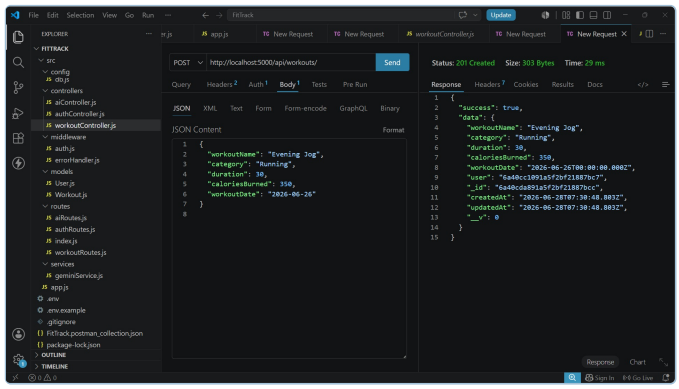
Register – 201 Created



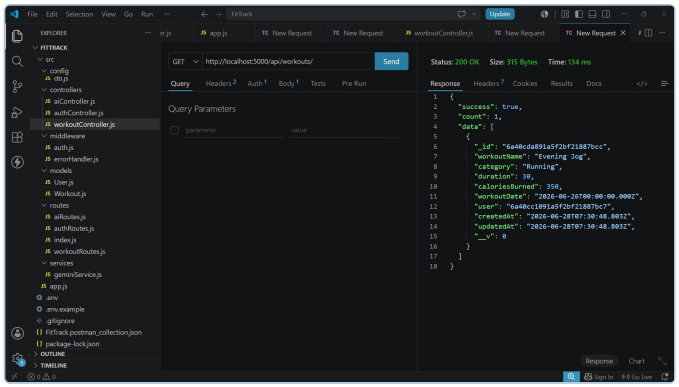
Login – 200 OK, JWT returned



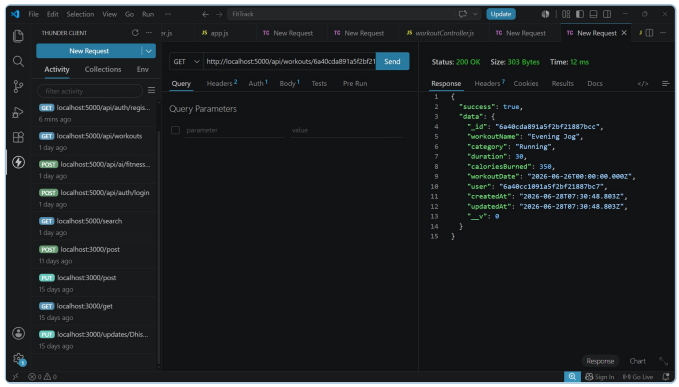
Get Profile – 200 OK



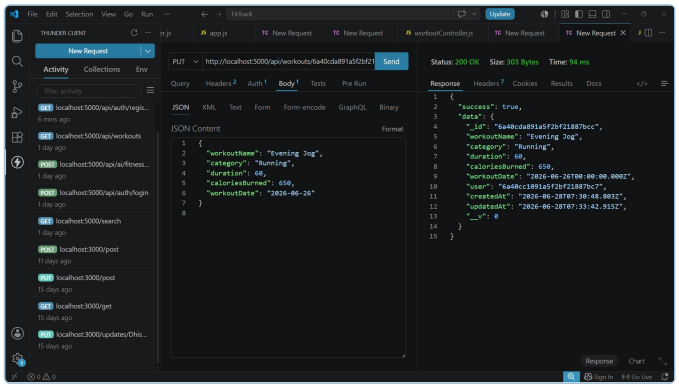
Add Workout – 201 Created



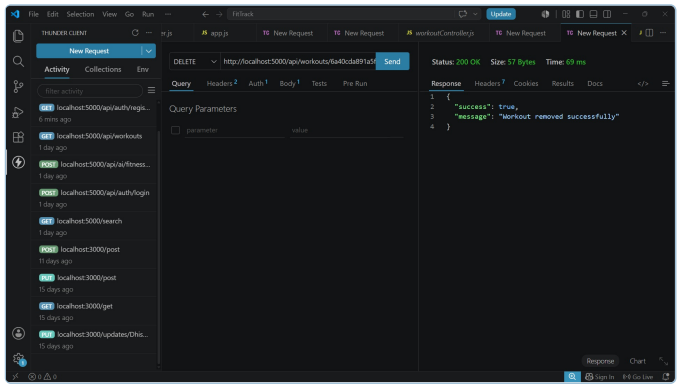
Get All Workouts – 200 OK



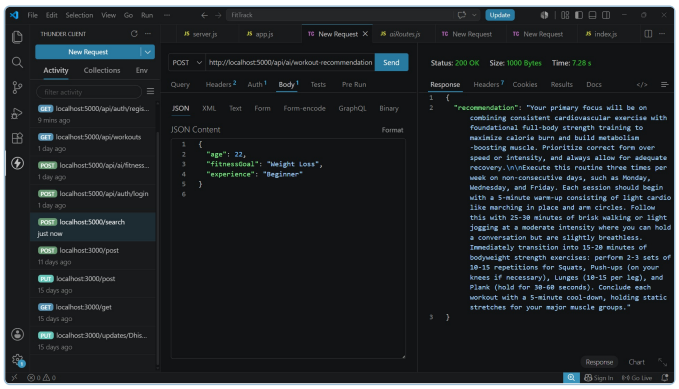
Get Workout by ID – 200 OK



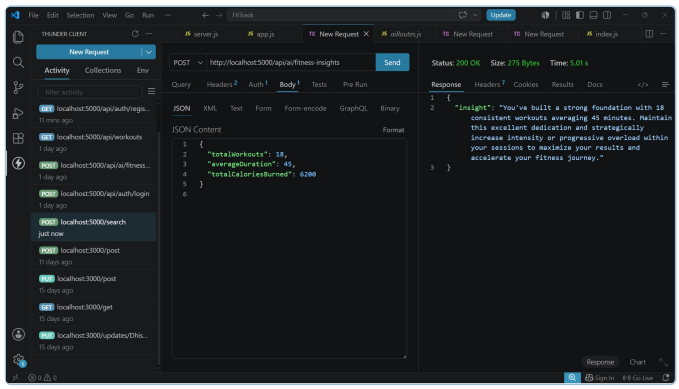
Update Workout – 200 OK



Delete Workout – 200 OK



AI Workout Recommendation – 200 OK (7.28s, Gemini latency)



AI Fitness Insights – 200 OK (5.01s, Gemini latency)

Demo & Repository

Resource	Link
GitHub Repository	https://github.com/thamaraiselvan6/fittrack_team_projt
Demo Video	Google Drive demo recording

Conclusion

AI Fit Track delivers a complete, secure, and AI-augmented workout-tracking backend within the 15–29 September 2026 cycle: authenticated CRUD workout management backed by validated MongoDB schemas, and two Google Gemini-powered endpoints that turn a user's own logged history into personalized recommendations and insights – directly resolving the fragmented, manual, and guidance-less logging experience identified in the team's initial problem statements.